

Name - Manisha
Section - ML2
Class Rollno - 32
University - 2419475

Assignment 1
System Design

Problem Statement: Build a small movie ticket booking system for a single cinema (Like PVR or INOX) as a menu driven C++ console program. A customer should be able to see which movie are playing, pick a show, see which seats are free, book seats, pay, get a ticket, and cancel a booking.

Step A

Ans

A. Functional Requirements

- FR1 - Movie listing: a customer requests the list of movies. The system shows, for each movie, its name, language, and duration. If no movies exist, an empty list is returned (not an error).
- FR2 - Show listing: Once a customer selects a movie, the system lists all shows for that movie (Screen + Start time). If the movie has no shows scheduled, an empty list is also shown, not an error.
- FR3 - Seat layout: Once the customer selects a show, the system displays the seat layout for that show's screen, organized by type (SILVER / GOLD / PLATINUM), with each seat marked AVAILABLE or BOOKED.
- FR4 (given) - Booking: a customer selects one or more seat numbers for a show. If any selected seat is already Booked, the whole booking is rejected and no seat changes state. Booking is confirmed only after payment succeeds.

- FR5 - Seat pricing : the system computes the price of each selected seat by its type (SILVER ₹150, GOLD ₹250, PLATINUM ₹400) and sums across all seats in the booking (any mix of types) to produce the total amount.
- FR6(given) - Payment : exactly one method (UPI / Card / Cash) per booking. If payment fails, seats are released and booking status becomes FAILED.
- FR7 - Ticket printing : after successful payment, the customer gets their ticket, containing booking ID, movie name, screen, show time, seats and total amount.
- FR8 - Booking cancellation : when a customer cancels a booking, its status becomes CANCELLED and all seats held by that booking return to available for that show. No actual monetary refund is simulated - only booking / seat state changes.

B. Non-functional Requirements:

- NFR1 (Modularity) - The system shall be implemented with one class per file, each class having a single, clearly defined responsibility.
- NFR2 (Extensibility) - Adding a new payment method (eg. Net Banking) shall require creating a new class that inherits from Payment, without modifying any existing class.

• NFR3 (Input Validation) - The system shall validate all user input and display a clear error message on invalid input, without crashing.

• NFR4 - (Reliability) - A failed payment shall never result in a confirmed booking. The temporarily selected seats shall be released if payment fails.

Step B

② Underline the nouns and verbs in the problem statement.
Then fill the Noun-Verb Analysis table. Rejecting a noun is also design - give a reason for rejecting it.

Arg	Noun	Keep as class?	Reason.
	Movie	Yes	• Has its own data (name, Language, duration)
	Seat	Yes	• Has number, type, price
	Seat Layout	NO	• It's just a view of a show's seats, not a stored thing.
	Show	Yes	• Has starting/ending time, and a screen it belongs to
	Screen	Yes	• Has different shows + movie details
	Ticket Cinema	Yes	• Represent the theatre and own its screens.
	Ticket	NO (corrected)	• No stored data - Just Ticket Printer's print() output.
	Payment Method	Yes	• Has payment method, amount deposited, total amount
	Screen Display	NO	• Just a screen display, not a class
	customer	Yes	• Has phone number, booking name
	Booking	Yes	• Has different member (id, show, seats, amount, status)

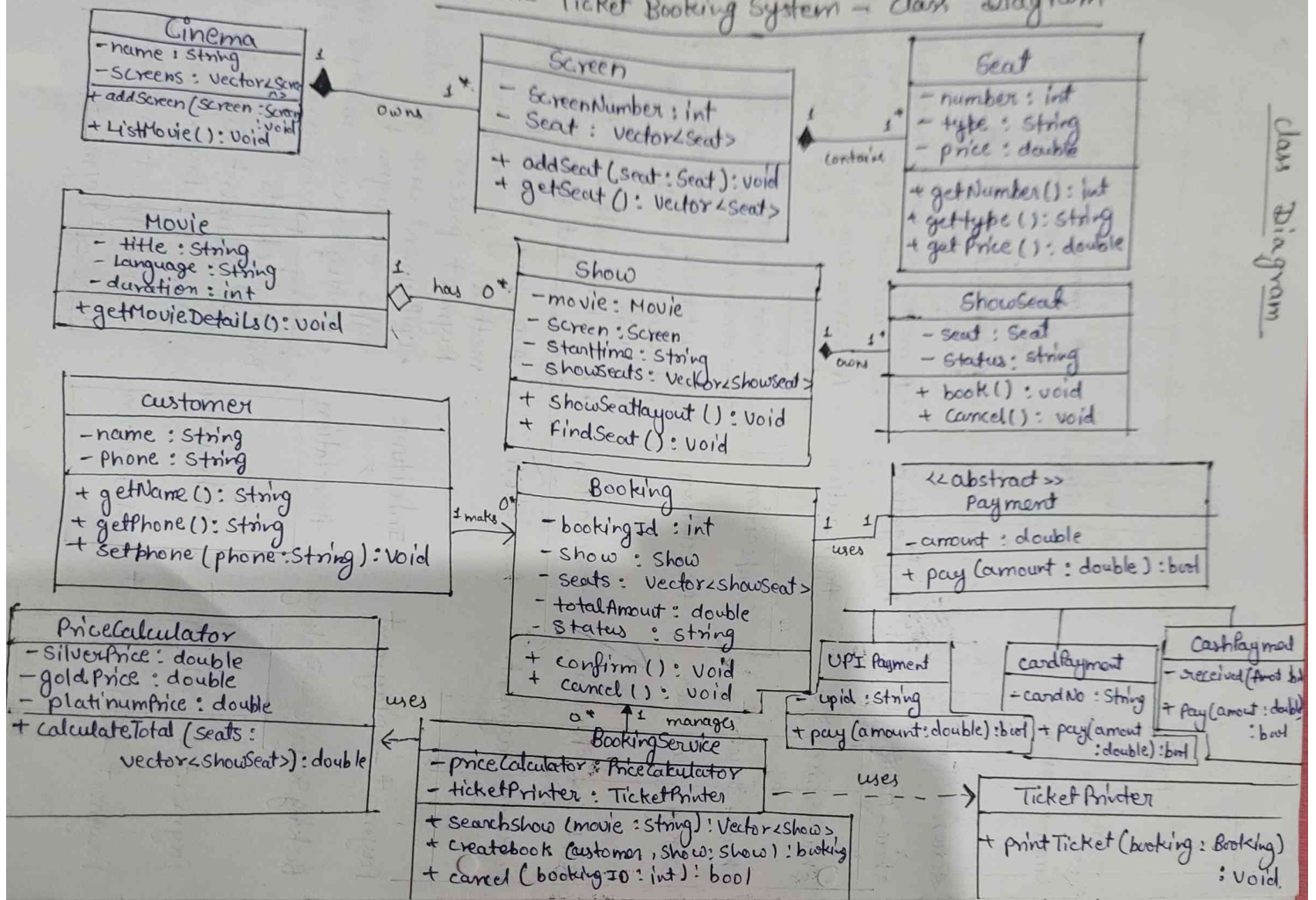
step C
 Ques 3. For each required class, state what it knows (data members), what it does (methods), and what it must not do.

class	Knows (Data)	Does (Methods)	Must Not Do.
Movie	title, language, duration	getMovieDetails()	Know about Shows or pricing
Seat	number, type, price	getPrice()	Track booking status.
Show	movie, screen, start time	ShowSeatLayout()	Handle payment
ShowSeat	seat, status	book(), cancel()	Calculate price.
Cinema	name, screens	addScreen(), List Movies()	Print tickets.
Customer	name, phone	getCustomerDetails()	change movie price.
Booking	booking ID, seats, amount, status	confirm(), cancel()	Print ticket.
Payment	Payment amount	pay(amount)	Calculate booking amount.
UPI Payment, card Payment, cash Payment.	Payment details.	pay(amount).	Manage seats or booking.
Price Calculator	Seat booked, seat type	Calculate total()	Book Seats
Ticket Printer	booking / ticket details	PrintTicket()	Modify booking data
Booking Service	booking-related objects / services	book(), cancel()	Directly choose a particular payment method.
Screen	Screen Number, seats	addSeat(), getSeats()	Handle booking or payment

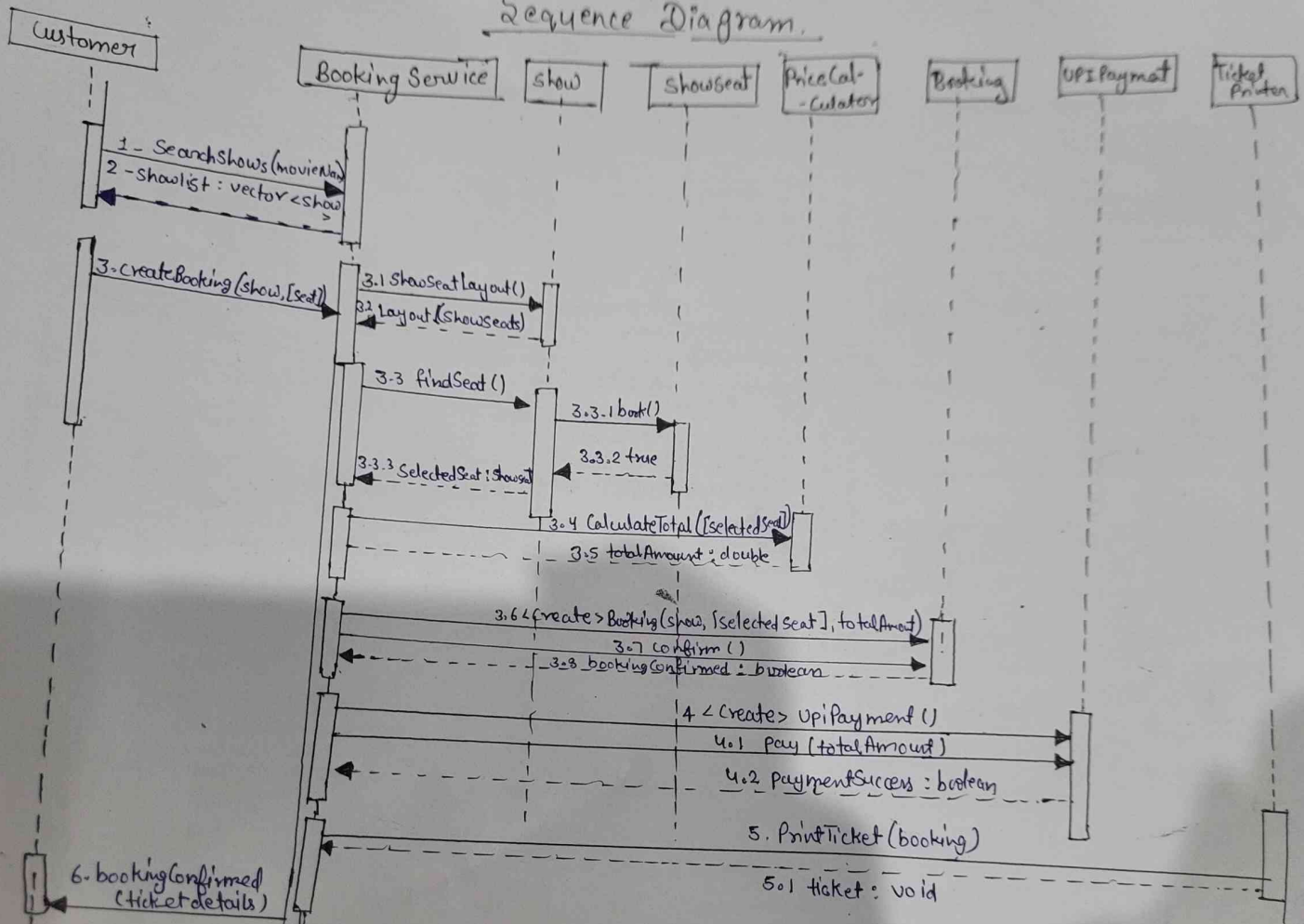
STEP D Relationship

<u>Pair</u>	<u>Your choice</u>	<u>Justification</u>
Cinema - Screen	Composition	• A Cinema owns its screens and screens are part of the Cinema.
screen - Seat	Composition	• A screen owns its seats, and seats are part of that screen.
show - Movie	Aggregation	• A movie can exist independently even without a particular show.
show - Screen	Aggregation	• A screen can exist independently and can be used for different shows.
show - showSeat	Composition	• Showseats belong to a particular show and exist for that show.
Booking - Customer	Association	• A customer can exist independently and may have multiple Bookings.
Booking - showSeat	Association	• A Booking refers to the selected showSeats but does not own them.
Booking - Payment	Association	• A Booking uses a Payment method to complete the Payment process.
Payment - upiPayment	Inheritance	• upiPayment is a specialized type of Payment.
BookingService - Booking	Association	• BookingService manages the booking process and works with Booking objects.
Payment - CardPayment	Inheritance	• CardPayment is a specialized type of Payment.
Payment - CashPayment	Inheritance	• CashPayment is a specialized type of Payment.

Movie Ticket Booking System - Class Diagram



Sequence Diagram



Step 4

Solid Principles

1. Single Responsibility Principle (SRP)

Each class in the system has a single Responsibility

- priceCalculator is responsible for calculating the total ticket price
- TicketPrinter is responsible for printing the ticket.
- BookingService is responsible for managing booking

and payment processing

therefore, the system follows the Single Responsibility Principle.

2. open/closed Principle (OCP)

The payment class acts as a base abstraction, and new payment methods can be added by creating new child classes.

Existing payment types are:

- upiPayment
- cardPayment
- cashPayment

for example: A new NetBankingPayment class can be added without modifying the existing BookingService class.

Therefore, the system follows the open/closed Principle.

3. Liskov Substitution Principle (LSP)

The child classes upiPayment, cardPayment and cashPayment can be used wherever a Payment object is expected.

All payment classes implement the pay() method.

Therefore the system follows the Liskov Substitution Principle.

④ Dependency Inversion Principle (DIP)

The BookingService class depends on the payment abstraction instead of depending directly on a specific payment method.

bool processpayment (Booking & booking, Payment & payment).

This allows different payment methods to be used without changing the BookingService class.

Therefore, the system follows the Dependency Inversion Principle.

⇒ Our Movie Ticket Booking system applies SRP, OCP, LSP and DIP principles. These principles make the system modular, flexible, extensible and easy to maintain.